॥वसुधैव कुटुम्बकम्॥

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE - Academic Year 2026-27

Java Programming – Lab (Batch 2025-29)

Lab Assignment No:- 24

Name of Student	Abobakr Mohamed Elameen
PRN No.	25070122015
Batch	2025-29
Class	Programming with JAVA
Academic Year & Semester	2nd Year – Sem III
Title of Assignment	Develop a JAVA GUI application integrated with JDBC to perform database operations.

Practice questions

1. Create a Library Management System GUI connected with database.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.*;

public class LibraryManagementSystem extends JFrame {


    private static final String URL  =
"jdbc:mysql://localhost:3306/testdb";
    private static final String USER = "root";
    private static final String PASS = "XXXXX";

    private JTextField txtId, txtTitle, txtAuthor, txtQty;
    private DefaultTableModel model;
    private JTable table;

    public LibraryManagementSystem() {
        setTitle("Library Management System");
        setSize(700, 500);
```

```
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));


        JPanel form = new JPanel(new GridLayout(4, 2, 5, 5));
        form.setBorder(BorderFactory.createTitledBorder("Book
Details"));
        txtId = new JTextField();
        txtTitle = new JTextField();
        txtAuthor = new JTextField();
        txtQty = new JTextField();
        form.add(new JLabel("Book ID:"));    form.add(txtId);
        form.add(new JLabel("Title:"));      form.add(txtTitle);
        form.add(new JLabel("Author:"));     form.add(txtAuthor);
        form.add(new JLabel("Quantity:"));   form.add(txtQty);



        JPanel buttons = new JPanel(new FlowLayout());
        JButton btnAdd = new JButton("Add");
        JButton btnUpdate = new JButton("Update");
        JButton btnDelete = new JButton("Delete");
        JButton btnClear = new JButton("Clear");
        buttons.add(btnAdd);
        buttons.add(btnUpdate);
        buttons.add(btnDelete);
        buttons.add(btnClear);

        JPanel top = new JPanel(new BorderLayout());
        top.add(form, BorderLayout.CENTER);
        top.add(buttons, BorderLayout.SOUTH);
        add(top, BorderLayout.NORTH);



        model = new DefaultTableModel(new String[]{"Book ID",
"Title", "Author", "Quantity"}, 0) {
            public boolean isCellEditable(int r, int c) { return
false; }
        };
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);



        table.getSelectionModel().addListSelectionListener(e -> {
            int r = table.getSelectedRow();
            if (r >= 0) {
                txtId.setText(model.getValueAt(r, 0).toString());
                txtTitle.setText(model.getValueAt(r,
1).toString());
```

```
                txtAuthor.setText(model.getValueAt(r,
2).toString());
                txtQty.setText(model.getValueAt(r, 3).toString());
            }
        });

        btnAdd.addActionListener(e -> addBook());
        btnUpdate.addActionListener(e -> updateBook());
        btnDelete.addActionListener(e -> deleteBook());
        btnClear.addActionListener(e -> clearFields());

        loadBooks();
    }

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }



    private void loadBooks() {
        model.setRowCount(0);
        try (Connection con = getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery("SELECT * FROM
books")) {
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("book_id"), rs.getString("title"),
                    rs.getString("author"), rs.getInt("quantity")
                });
            }
        } catch (SQLException ex) {
            showError(ex);
        }
    }



    private void addBook() {
        if (!validateInput()) return;
        String sql = "INSERT INTO books (book_id, title, author,
quantity) VALUES (?, ?, ?, ?)";
        try (Connection con = getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setInt(1,
Integer.parseInt(txtId.getText().trim()));
            ps.setString(2, txtTitle.getText().trim());
            ps.setString(3, txtAuthor.getText().trim());
            ps.setInt(4,
Integer.parseInt(txtQty.getText().trim()));
```

```
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Book added
successfully.");
            loadBooks();
            clearFields();
        } catch (SQLException ex) {
            showError(ex);
        }
    }

    // UPDATE
    private void updateBook() {
        if (!validateInput()) return;
        String sql = "UPDATE books SET title=?, author=?,
quantity=? WHERE book_id=?";
        try (Connection con = getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setString(1, txtTitle.getText().trim());
            ps.setString(2, txtAuthor.getText().trim());
            ps.setInt(3,
Integer.parseInt(txtQty.getText().trim()));
            ps.setInt(4,
Integer.parseInt(txtId.getText().trim()));
            int rows = ps.executeUpdate();
            JOptionPane.showMessageDialog(this, rows > 0 ? "Book
updated." : "No book found with that ID.");
            loadBooks();
        } catch (SQLException ex) {
            showError(ex);
        }
    }

    // DELETE
    private void deleteBook() {
        if (txtId.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "Select a book or
enter Book ID to delete.");
            return;
        }
        int choice = JOptionPane.showConfirmDialog(this, "Delete
this book?", "Confirm", JOptionPane.YES_NO_OPTION);
        if (choice != JOptionPane.YES_OPTION) return;
        String sql = "DELETE FROM books WHERE book_id=?";
        try (Connection con = getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setInt(1,
Integer.parseInt(txtId.getText().trim()));
            int rows = ps.executeUpdate();
```

```
            JOptionPane.showMessageDialog(this, rows > 0 ? "Book
deleted." : "No book found with that ID.");
            loadBooks();
            clearFields();
        } catch (SQLException | NumberFormatException ex) {
            showError(ex);
        }
    }

    private boolean validateInput() {
        if (txtId.getText().trim().isEmpty() ||
txtTitle.getText().trim().isEmpty()
                || txtAuthor.getText().trim().isEmpty() ||
txtQty.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "All fields are
required.");
            return false;
        }
        try {
            Integer.parseInt(txtId.getText().trim());
            Integer.parseInt(txtQty.getText().trim());
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Book ID and
Quantity must be numbers.");
            return false;
        }
        return true;
    }

    private void clearFields() {
        txtId.setText("");
        txtTitle.setText("");
        txtAuthor.setText("");
        txtQty.setText("");
        table.clearSelection();
    }

    private void showError(Exception ex) {
        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage(), "Database Error", JOptionPane.ERROR_MESSAGE);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
 LibraryManagementSystem().setVisible(true));
    }
}
```

2. Create a Book Issue Tracking System GUI application using Java Swing connected to a database. The system should store book ID, student name, issue date, and return date, and allow record management.

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.sql.*;

public class BookIssueTrackingSystem extends JFrame {

    private static final String URL  =
"jdbc:mysql://localhost:3306/testdb";
    private static final String USER = "root";
    private static final String PASS = "XXXXX";

    private JTextField txtIssueId, txtBookId, txtStudent,
txtIssueDate, txtReturnDate;
    private DefaultTableModel model;
    private JTable table;

    public BookIssueTrackingSystem() {
        setTitle("Book Issue Tracking System");
        setSize(750, 520);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));


        JPanel form = new JPanel(new GridLayout(5, 2, 5, 5));
        form.setBorder(BorderFactory.createTitledBorder("Issue
Details"));
        txtIssueId = new JTextField();
        txtIssueId.setEditable(false);
        txtBookId = new JTextField();
        txtStudent = new JTextField();
        txtIssueDate = new JTextField();
        txtReturnDate = new JTextField();
        form.add(new JLabel("Issue ID
(auto):"));            form.add(txtIssueId);
        form.add(new JLabel("Book
ID:"));                    form.add(txtBookId);
        form.add(new JLabel("Student
Name:"));               form.add(txtStudent);
        form.add(new JLabel("Issue Date (yyyy-MM-
dd):"));    form.add(txtIssueDate);
```

```
        form.add(new JLabel("Return Date (yyyy-MM-
dd):"));    form.add(txtReturnDate);



        JPanel buttons = new JPanel(new FlowLayout());
        JButton btnIssue = new JButton("Issue Book");
        JButton btnUpdate = new JButton("Update / Mark Returned");
        JButton btnDelete = new JButton("Delete");
        JButton btnClear = new JButton("Clear");
        buttons.add(btnIssue);
        buttons.add(btnUpdate);
        buttons.add(btnDelete);
        buttons.add(btnClear);

        JPanel top = new JPanel(new BorderLayout());
        top.add(form, BorderLayout.CENTER);
        top.add(buttons, BorderLayout.SOUTH);
        add(top, BorderLayout.NORTH);



        model = new DefaultTableModel(
                new String[]{"Issue ID", "Book ID", "Student
Name", "Issue Date", "Return Date"}, 0) {
            public boolean isCellEditable(int r, int c) { return
false; }
        };
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);



        table.getSelectionModel().addListSelectionListener(e -> {
            int r = table.getSelectedRow();
            if (r >= 0) {
                txtIssueId.setText(String.valueOf(model.getValueAt
(r, 0)));
                txtBookId.setText(String.valueOf(model.getValueAt(
r, 1)));
                txtStudent.setText(String.valueOf(model.getValueAt
(r, 2)));
                txtIssueDate.setText(String.valueOf(model.getValue
At(r, 3)));
                Object ret = model.getValueAt(r, 4);
                txtReturnDate.setText(ret == null ? "" :
ret.toString());
            }
        });

        btnIssue.addActionListener(e -> issueBook());
        btnUpdate.addActionListener(e -> updateRecord());
```

```
        btnDelete.addActionListener(e -> deleteRecord());
        btnClear.addActionListener(e -> clearFields());

        loadRecords();
    }

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }


    private void loadRecords() {
        model.setRowCount(0);
        try (Connection con = getConnection();
             Statement st = con.createStatement();
             ResultSet rs = st.executeQuery("SELECT * FROM
book_issue ORDER BY issue_id")) {
            while (rs.next()) {
                model.addRow(new Object[]{
                    rs.getInt("issue_id"), rs.getInt("book_id"),
rs.getString("student_name"),
                    rs.getDate("issue_date"),
rs.getDate("return_date")
                });
            }
        } catch (SQLException ex) {
            showError(ex);
        }
    }


    private void issueBook() {
        if (!validateInput()) return;
        String sql = "INSERT INTO book_issue (book_id,
student_name, issue_date, return_date) VALUES (?, ?, ?, ?)";
        try (Connection con = getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setInt(1,
Integer.parseInt(txtBookId.getText().trim()));
            ps.setString(2, txtStudent.getText().trim());
            ps.setDate(3,
Date.valueOf(txtIssueDate.getText().trim()));
            setReturnDate(ps, 4);
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Book issued
successfully.");
            loadRecords();
            clearFields();
        } catch (SQLException ex) {
```

```
            showError(ex);
        }
    }


    private void updateRecord() {
        if (txtIssueId.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "Select a record
from the table first.");
            return;
        }
        if (!validateInput()) return;
        String sql = "UPDATE book_issue SET book_id=?,
student_name=?, issue_date=?, return_date=? WHERE issue_id=?";
        try (Connection con = getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setInt(1,
Integer.parseInt(txtBookId.getText().trim()));
            ps.setString(2, txtStudent.getText().trim());
            ps.setDate(3,
Date.valueOf(txtIssueDate.getText().trim()));
            setReturnDate(ps, 4);
            ps.setInt(5,
Integer.parseInt(txtIssueId.getText().trim()));
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Record
updated.");
            loadRecords();
            clearFields();
        } catch (SQLException ex) {
            showError(ex);
        }
    }


    private void deleteRecord() {
        if (txtIssueId.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "Select a record
from the table first.");
            return;
        }
        int choice = JOptionPane.showConfirmDialog(this, "Delete
this record?", "Confirm", JOptionPane.YES_NO_OPTION);
        if (choice != JOptionPane.YES_OPTION) return;
        String sql = "DELETE FROM book_issue WHERE issue_id=?";
        try (Connection con = getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setInt(1,
Integer.parseInt(txtIssueId.getText().trim()));
```

```
            ps.executeUpdate();
            JOptionPane.showMessageDialog(this, "Record
deleted.");
            loadRecords();
            clearFields();
        } catch (SQLException ex) {
            showError(ex);
        }
    }

    private void setReturnDate(PreparedStatement ps, int index)
throws SQLException {
        String r = txtReturnDate.getText().trim();
        if (r.isEmpty()) ps.setNull(index, Types.DATE);
        else ps.setDate(index, Date.valueOf(r));
    }

    private boolean validateInput() {
        if (txtBookId.getText().trim().isEmpty() ||
txtStudent.getText().trim().isEmpty()
                || txtIssueDate.getText().trim().isEmpty()) {
            JOptionPane.showMessageDialog(this, "Book ID, Student
Name and Issue Date are required.");
            return false;
        }
        try {
            Integer.parseInt(txtBookId.getText().trim());
            Date.valueOf(txtIssueDate.getText().trim());
            if (!txtReturnDate.getText().trim().isEmpty())
                Date.valueOf(txtReturnDate.getText().trim());
        } catch (IllegalArgumentException ex) {
            JOptionPane.showMessageDialog(this, "Book ID must be a
number and dates must be in yyyy-MM-dd format.");
            return false;
        }
        return true;
    }

    private void clearFields() {
        txtIssueId.setText("");
        txtBookId.setText("");
        txtStudent.setText("");
        txtIssueDate.setText("");
        txtReturnDate.setText("");
        table.clearSelection();
    }

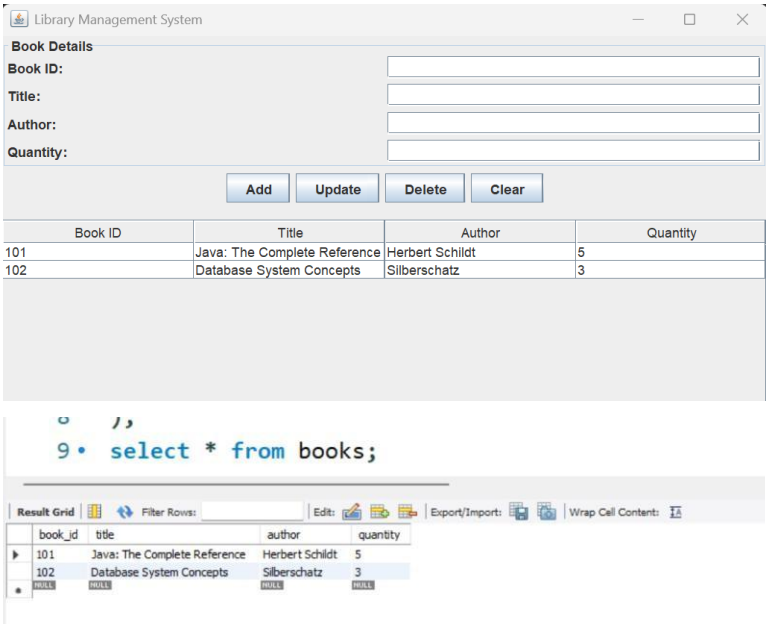
    private void showError(Exception ex) {
```

```
        JOptionPane.showMessageDialog(this, "Error: " +
ex.getMessage(), "Database Error", JOptionPane.ERROR_MESSAGE);
    }

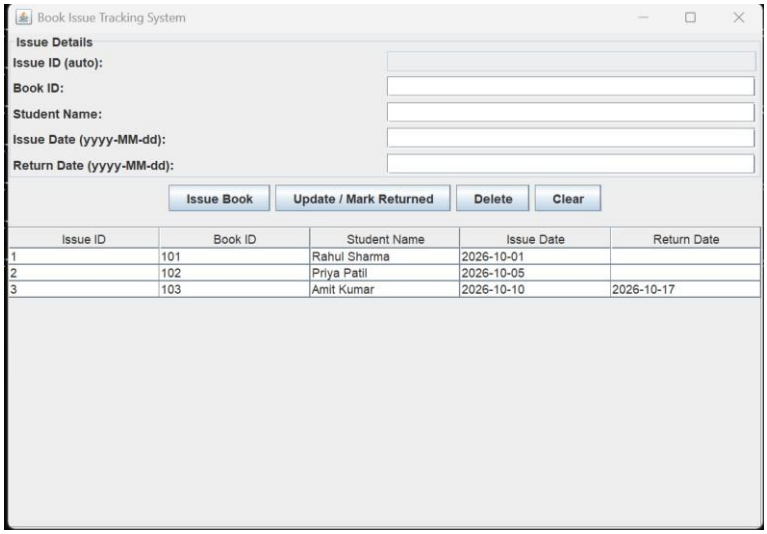
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
 BookIssueTrackingSystem().setVisible(true));
    }
}
```

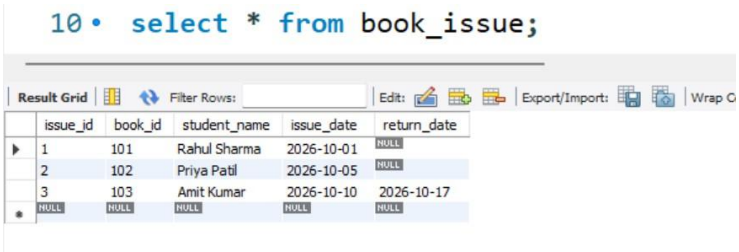
Output

1)



2)



	
Conclusion	Thus, we have successfully developed Java GUI applications using Swing and integrated them with a database using JDBC. The programs perform insert, update, delete and display operations on database records through a graphical interface, which makes database handling simple and user-friendly.